



Writeup

CURSO:
SEGURIDAD DE SOFTWARE

REMEETING THE WHEEL (MEDIUM)

Profesor:
Natán Fernández de Castro

Estudiantes
Kener Adiel Castillo
Alejandro Cerdas Vargas

Carnet
2023179846
2023073037

23 de junio de 2026

Índice

1. Información de Contacto	2
2. Resumen ejecutivo	2
2.1. Alcance	2
2.2. Hallazgos y recomendaciones	2
3. Información general	2
4. Reconocimiento	3
4.1. Recopilación inicial de información	3
4.2. Elementos identificados	4
5. Enumeración y análisis	6
5.1. Hallazgos	6
5.2. Posibles vectores de ataque	8
5.3. Caminos probados	8
6. Explotación	9
6.1. Vulnerabilidades identificadas	9
6.2. Pasos para explotación	10
7. Evidencia de resolución	16
8. Remediación	17
9. Lecciones aprendidas	18
10. Acrónimos	18
11. Recursos consultados	18

1. Información de Contacto

Nombre	Correo Electrónico	Número de Teléfono
Alejandro Cerdas Vargas	al.cerdas@estudiantec.cr	506-8403-4568
Kener Adiel Castillo	kencastillo@estudiantec.cr	506-7020-8452

Cuadro 1: Información de los testers

2. Resumen ejecutivo

2.1. Alcance

Este challenge de dificultad media consistió en encontrar las vulnerabilidades que tenía un script de Python que implementaba el algoritmo de encriptación Rivest, Shamir y Adleman ([RSA](#)) junto con Advanced Encryption Standard ([AES](#)) para realizar el intercambio de mensajes de forma segura y eficiente. El objetivo fue descifrar la llave que utilizaba el algoritmo [AES](#) para descifrar y obtener el mensaje transmitido por medio del análisis de los mensajes compartidos entre dos puntos.

2.2. Hallazgos y recomendaciones

En el estudio del challenge, se descubrió una vulnerabilidad en el método que se usaba para generar la llave del cifrado [AES](#), debido a que la llave se generaba a partir de dos números pequeños, los cuales era posible computar. De esta forma, se puede obtener la llave y ser capaz de descifrar todos los datos en el intercambio de información. Producto de esto, se recomienda usar otro método para generar la llave para el cifrado [AES](#), así como aumentar el tamaño de la llave usada, para evitar que pueda ser computada en un tiempo razonable. Adicionalmente, se recomienda usar un cifrado [RSA](#) de mayor tamaño, ya que la versión usada es considerada hoy en día, vulnerable. Finalmente, se recomienda implementar principios de seguridad como usar estándares criptográficos y hacer uso de bibliotecas y herramientas que hayan sido certificadas como ideales para seguridad.

3. Información general

El challenge resuelto para este write-up pertenece a la plataforma Hack The Box ([HTB](#)). Más detalles en la tabla [2](#).

Task	ReMeeting the Wheel
Dificultad	Medium
Categoria	Cripto
Fecha de Resolución	28 de mayo del 2026
Usuario	aleCaleCaleC y kencast

Cuadro 2: Información General

4. Reconocimiento

4.1. Recopilación inicial de información

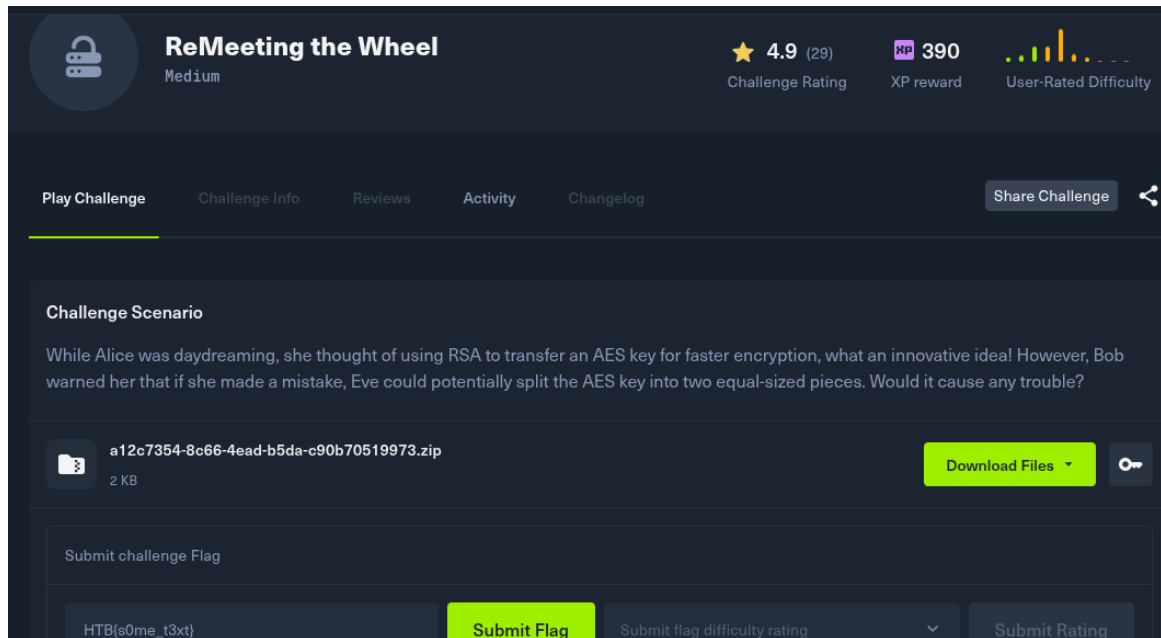


Figura 1: El challenge en HTB

Para iniciar el challenge hay que descargar el archivo `a12c7354-8c66-4ead-b5da-c90b70519973.zip` en HTB a como se muestra en la figura 1.

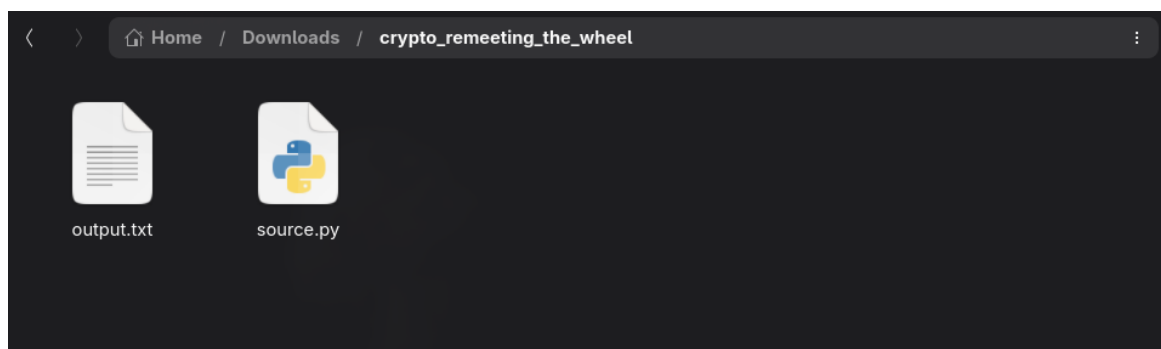


Figura 2: Archivos del challenge

Una vez descargado, hay que descomprimir el archivo zip, en HTB se encuentra la contraseña. En total se cuenta con dos archivos, uno de texto `output.txt` y un script del lenguaje Python `source.py`, como se observa en 2. Esto sugiere que para el challenge hay que analizar el código de Python y su salida en busca de vulnerabilidades. Como sugiere la categoría del challenge y la descripción, este está relacionado con el algoritmo [RSA](#) y [AES](#), puede consultar sobre estos en [1, 4]. También se menciona algo de dividir la llave de [AES](#) en dos partes de igual tamaño.

4.2. Elementos identificados

Como no se cuenta con ninguna red, computadora o sistema objetivo, no se utilizó ninguna herramienta para el análisis de puertos u otros elementos relacionados. Solo se cuenta con el script y la salida de este, así que los elementos identificados son los siguientes:

Elemento	Tipo	Descripción
bytes_to_long	Función	Pertenece al módulo de Python <code>Crypto.Util.number</code> . Convierte un string de bytes a un número. [2]
getPrime	Función	Pertenece al módulo de Python <code>Crypto.Util.number</code> . Retorna un número primo random de N bits. [2]
FLAG	Variables	Hilera de bytes. Pertenece al módulo <code>Secret</code> , que parece ser definido por el creador.
sha256	Función	Pertenece al módulo <code>hashlib</code> . Retorna un hash de tamaño 256.
RSA	Algoritmo	Método de encriptación asimétrica muy usado [1].
randint	Función	Genera números enteros en rango [11]
AES	Clase	Implementa el cifrado simétrico por bloque [5]
pad	Función	Aplica “padding” estándar [7]

Cuadro 3: Elementos identificados en los archivos

Las bibliotecas que usan parecen ser seguras y aptas para procesos criptográficos, por lo que estas no son el enfoque. El script representa más que todo la implementación parcial del algoritmo [RSA](#) y del [AES](#). En [3] hay una explicación más detallada del algoritmo [RSA](#), pero en general se resume en las fases de:

Generación de llaves:

Se eligen dos números primos grandes p y q , estos son secretos. Se calcula $n = pq$ como parte de la llave pública y privada. También, $\phi(n) = (p - 1)(q - 1)$ ya que p y q son números primos. Se elige $e : 1 < e < \phi(n) : \gcd(e, \phi(n)) = 1$, de esta manera son coprimos. Finalmente, se calcula un d tal que $d \times e \equiv 1 \pmod{\phi(n)}$. La llave pública se representa por (n, e) y la privada por (n, d) .

Cifrado:

Si M es el mensaje representado de forma numérica. $C = M^e \pmod n$ es el mensaje cifrado.

Descifrado:

El mensaje original es $M = C^d \pmod n$.

A su vez el algoritmo de cifrado [AES](#), es un derivado del algoritmo Rijndael. [AES](#) opera bloques de 16 bytes en una matriz de 4×4 bytes. A esta matriz se le aplica cuatro operaciones básicas:

- **SubBytes** realiza una sustitución de cada byte de la matriz por uno correspondiente de una tabla fija.
- **ShiftRows** cada fila le realiza una rotación un número de veces.
- **MixColumns** cada columna la multiplica por un polinomio constante con el fin de realizarle una transformación lineal.
- **AddRoundKey** cada byte de la matriz se opera con un byte de una subclave, mediante el XOR.

Este procesos se lleva a cabo durante varios “rounds” [\[4\]](#).

5. Enumeración y análisis

5.1. Hallazgos

```
class RSAGen: 1 usage
    def __init__(self, bits):
        self.bits = bits

    def keygen(self): 1 usage
        p = getPrime(self.bits//2)
        q = getPrime(self.bits//2)
        self.n = p*q
        self.e = 0x10001
        phi = (p-1)*(q-1)
        self.d = pow(self.e, -1, phi)
        return (self.n, self.e), (p, q, phi, self.d)

    def encrypt(self, m): 1 usage
        return pow(m, self.e, self.n)

    def decrypt(self, c):
        return pow(c, self.d, self.n)
```

Figura 3: Código del [RSA](#)

En la figura 3 se define la clase `RSAGen` con la implementación estándar del algoritmo [RSA](#), en este caso se usan dos primos de igual cantidad de bits y se define `e=65537`.

```

class AESGen: 1 usage
    def __init__(self, size):
        key1 = randint(1 << size, 1 << (size+1))
        key2 = randint(1 << size, 1 << (size+1))
        self.k = key1 * key2
        assert 40 <= self.k.bit_length() <= 42
        self.KEY = sha256(str(self.k).encode()).digest()

    def encrypt(self, m): 1 usage
        cipher = AES.new(self.KEY, AES.MODE_ECB)
        enc_secret = cipher.encrypt(pad(m, block_size: 16))
        return enc_secret

```

Figura 4: Código del AES

En la figura 4, se observa que se inicializa la clase de AESGen con la entrada de `size`, tamaño que posteriormente sería usado para generar `key1` y `key2` con `randint`. Se puede observar que $2^{size} \leq key1, key2 \leq 2^{size+1}$. Con el producto de ambas llaves se define `k` y tras aplicarle el sha256 a este `k` se genera `KEY`. Finalmente, en la función de `encrypt` se usa `KEY` para cifrar el mensaje con un padding de 16.

```

def main(): 1 usage
    rsa = RSAGen(1024)
    aes = AESGen(20)
    pubkey, _ = rsa.keygen()
    enc_aes_key = l2b(rsa.encrypt(aes.k))
    enc_secret = aes.encrypt(FLAG)

    with open('output.txt', 'w') as f:
        f.write("Bob :: Hi Alice, here is my public key:\n")
        f.write(f"{{pubkey[0]}}, {{pubkey[1]}}\n")
        f.write("Alice :: Hi Bob, here is my encrypted AES key, don't forget to sha256-hash it!\n")
        f.write(f"Encrypted AES Key = {{enc_aes_key.hex()}}\n")
        f.write("Bob :: Got it, here is the encrypted secret you requested:\n")
        f.write(f"Encrypted Secret = {{enc_secret.hex()}}")

```

Figura 5: Código principal

En la figura 5 se inicializa la clase RSAGen de 1024 bits (números primos de 512 bits) y la clase AESGen con 21 bits (factores de la llave de tamaño 21 bits). Después se genera la llave pública de Bob con `rsa.keygen()` y se imprime. La llave pequeña de Alice para AES se encripta con `rsa.encrypt(aes.k)` utilizando la llave pública de Bob y se imprime en formato hexadecimal. Por último, se encripta el mensaje `aes.encrypt(FLAG)` con el algoritmo AES utilizando la llave de 256

bits y el resultado se imprime en formato hexadecimal. En resumen, la información que tiene es:

Dato	Función
Llave pública de RSA	Encriptar la llave del AES
Llave encriptada	Llave encriptada con RSA que se usó para cifrar el mensaje. Se busca descifrar esta.
Mensaje cifrado	Mensaje cifrado con AES , se busca descifrar este.

Cuadro 4: Datos que se tienen

Lo que llama la atención principalmente de lo analizado anteriormente es que se usa [RSA](#)-1024, lo cual se considera inseguro y no se recomienda por ser susceptible a ataques de fuerza bruta. Además, se usa [AES](#)-256, pero para definir la llave se multiplican dos números de 20 bits cada uno y al resultado se le aplica un hash de 256 bits. Estos factores de la llave son muy pequeños y hacen posible un ataque de fuerza bruta para encontrar los dos factores que la componen.

5.2. Posibles vectores de ataque

En [HTB](#), la bandera esperada corresponde a un texto de la forma `HTB{ }` con cierto contenido entre ambas llaves, y tomando en cuenta la información que se tiene, el objetivo es descifrar el mensaje. Con la información antes recaudada, se sabe que se tiene el módulo y el exponente usados para encriptar la llave usada en el [AES](#) y adicionalmente, se tiene el resultado tras la acción del [RSA](#).

Un posible vector de ataque es realizar “brute force” sobre ambos valores de `key1` y `key2` realizando ciertas optimizaciones, o iterar sobre `k1` y obtener `k2` por medio de algún despeje de la ecuación usada por [RSA](#) para cifrar. La principal es usar la técnica “Meet-in-the-Middle”, la cual plantea dividir el espacio de búsqueda en dos espacios más pequeños que se complementen para resolver el problema. Pasando de una complejidad de $O(n^2)$ a una de $O(n \log_2(n))$, donde $n = 2^{20}$ es el espacio de búsqueda.

5.3. Caminos probados

Lo primero que se puede pensar en estas situaciones con [RSA](#) es si existe alguna forma de descifrar la llave de forma directa. Al tener el módulo, lo único que sería necesario es obtener el d , de la figura 3. Es fácil darse cuenta que, debido a que no se comparte este d , la única forma de obtenerlo sería mediante el ϕ de la figura 3. Para esto habría que factorizar n en dos primos de 512 bits. De lograr esto, ocupa una capacidad de cómputo exagerada, por lo tanto este camino no es viable.

Lo segundo es intentar realizar un proceso de fuerza bruta sobre la llave del [AES](#). Si bien se podría intentar sobre todas las posibilidades de `key1` y `key2`. Esto tomaría $2^{20} \times 2^{20} \times \log_2(n) \approx 10^{14}$, lo que representa $\approx 10^6$ segundos o ≈ 115 días, provocando que no fuera posible entregar este informe a tiempo. Por último, si se itera sobre `k1` tampoco se puede obtener de forma eficiente `k2` por medio de un despeje directo, por lo que se ocupa algo más.

6. Explotación

6.1. Vulnerabilidades identificadas

ID	Nombre	Descripción
CWE-326	Inadequate Encryption Strength	El producto transmite datos usando un método criptográfico fuerte teóricamente, pero vulnerable a ataques de fuerza bruta.
CWE-522	Insufficiently Protected Credentials	El producto transmite o guarda credenciales (o llaves) pero usa un método inseguro susceptible a la recuperación de estos.

Cuadro 5: Principales vulnerabilidades encontradas en la fuente CWE [13]

ID	Nombre	Severidad	Descripción
CAPEC-97	Cryptanalysis	Muy alta	Encontrar vulnerabilidades en algoritmos criptográficos para descifrar un mensaje o la llave privada.

Cuadro 6: Principales patrones de ataque encontrados en la fuente CAPEC [14]

ID	Nombre	CVSS	Descripción
CVE-2026-5363	Use of weak cryptographic key in TP-Link Archer C7	Medium	Encriptación de contraseñas usando RSA-1024. Un atacante puede interceptar el mensaje y hacer un ataque de fuerza bruta o de factorización para encontrar la clave privada.
CVE-2025-4876	ConnectWise LLC	Medium	Le permite extraer la llave del AES “alambrada” mediante ingeniería inversa

Cuadro 7: Ataques relacionados encontrados en la plataforma CVE [15]

En la tabla 5 se identifican las principales vulnerabilidades relacionadas con el challenge resuelto. La vulnerabilidad CWE-326 se aplica en este caso, ya que se identificó que se estaba usando RSA-1024 para encriptar la llave, esto vuelve vulnerable al algoritmo a un ataque de fuerza bruta como la factorización de la clave pública para descifrar el mensaje. Sin embargo, para llevar a cabo esto se requiere mucha capacidad de cómputo. En la tabla 7 se identifica un ataque de severidad media que se relaciona con esta vulnerabilidad.

La anterior vulnerabilidad fue la que permitió resolver el challenge, ya que los algoritmos de cifrado RSA y AES son métodos que se consideran seguros; sin embargo, una insegura generación de la llave para el algoritmo AES permite un ataque de fuerza bruta para encontrar la llave, lo cual está relacionado también con la vulnerabilidad CWE-522.

En general, el patrón de ataque que aplica en este caso es CAPEC-97, especificado en la tabla 6. El algoritmo RSA y AES no son los que tenían la vulnerabilidad, sino la forma en la que se generan y se comparten las llaves, lo que permite obtener información clave de esta y reducir el espacio de búsqueda considerablemente. Por otro lado, no se encontró un ataque específico (CVE) que represente la forma en que se resolvió este challenge, por lo que se trata más de una vulnerabilidad en un caso particular.

Respecto a la relación con los contenidos del curso, estas vulnerabilidades y patrones de ataques corresponden al reino de ‘Criptografía’ vistos en la sección ‘2.1. Taxonomía de los patrones de ataque’ relacionados con el algoritmo RSA y AES. El patrón de ataque consistió en el análisis matemático y criptográfico para obtener información del algoritmo y las llaves usadas.

6.2. Pasos para explotación

Analizando el código en la figura 5:

$$K = k_1 \times k_2 \quad k_1, k_2 \leq 2^{21} \quad (1)$$

Además la forma de cifrado en la figura 3:

$$M \equiv K^e (n) \quad (2)$$

$$K^e \equiv (k_1 \times k_2)^e \equiv k_1^e \times k_2^e (n) \quad (3)$$

$$k_2^e \equiv M \times (k_1^e)^{-1} (n) \quad (4)$$

La ecuación anterior está bien definida, debido a que k_1^e siempre es coprimo con n ya que $n = pq$: $p, q \geq 2^{512} \wedge k_1 \leq 2^{41}$, por lo tanto el inverso modular existe.

De esta forma, se puede concluir que si se realiza fuerza bruta para calcular k_1 y k_2 que cumplan la congruencia, se podría calcular la llave mediante el producto de estos dos. Como ya se mencionó, tener dos “for” anidados de 2^{20} a 2^{21} resultaría en un tiempo poco deseable, y sacar la raíz e –ésima modular es casi imposible. La observación clave es que se puede separar la búsqueda de ambos con la técnica “Meet in the Middle” ya que el espacio de búsqueda de k_1 y k_2 es simétrico y $\sqrt{K} \leq 2^{21}$.

```

1  # Imports
2  from Crypto.Util.number import bytes_to_long
3  from Crypto.Util.number import inverse
4  from Crypto.Cipher import AES
5  from Crypto.Util.Padding import unpad
6  from hashlib import sha256
7
8  # Valores en hexadecimal dados
9  hex_key = '4da0230d2b8b46e3a7065f32c46df019739cc002a208cc37767a82c3e94edfc3440fa4b24a32274e35d5ddceaa33505c4f2a57281c3a5d6d4db3a0dbdbb30dbf3732411'
10 hex_msg = 'ce8f36aa844ab00319bcd4f86460a10d77492c060b2c2a91615f4cd1f2d0702e76b68f1ec0f11d15704ba52c5dacc60018d5ed87368464acd030ce6230efdbff7b18cbe'
11
12 # Obtener su valor sus bytes
13 byte_key = bytes.fromhex(hex_key)
14 aes_msg = bytes.fromhex(hex_msg)
15
16 # Convertir la llave a long
17 enc_key = bytes_to_long(byte_key)
18
19 # Imprimir la llave encriptada
20 print(enc_key)
21
22 # Llave pública RSA
23 n = 727414232080334054034922756987629689365146573148234424854535598701052001183394059008582999987470612784867033440738722844402907006053822044
24 e = 65537
25
26 # Diccionario para guardar valores encontrados
27 dic = {}
28
29 # Límites para las llaves de [2^20, 2^21]
30 L=1<<20
31 R=1<<21
32

```

Figura 6: Fragmento de código donde se transforman los datos otorgados

The screenshot shows a code editor window titled 'crypto_remeeting_the_wheel'. The left sidebar contains a file explorer with folders like 'venv' and files like 'ag.py', 'output.txt', 'parejas.txt', and 'source.py'. The main editor area displays the same Python code as in Figure 6. At the bottom, there is a terminal window showing the command to run the script: `/home/alejandros/Descargas/crypto_remeeting_the_wheel/.venv/bin/python /home/alejandros/Descargas/crypto_remeeting_the_wheel/ag.py`, followed by a long hexadecimal output string.

Figura 7: Valor numérico de la llave de AES cifrada con RSA

La primera parte del código de la figura 6, muestra los imports necesarios para el proceso de descifrado. Seguido, en las líneas 9 y 10, se “alambran” los valores en hexadecimal de la llave de AES y el mensaje cifrados, recaudados del archivo `output.txt`. Posteriormente, se llama a la función `fromhex` (líneas 13 y 14) la cual devuelve los bytes de la llave y el mensaje cifrados. Después, en la línea 17, se imprime el valor numérico de la llave tras la función `bytes_to_long()`. En las líneas 23 y

24, se definen el n y e extraídos del archivo `output.txt`. En la línea 27 se define un diccionario y en las líneas 30 y 31, los límites de k_1 y k_2 . En la figura 7, se observa el resultado del print de la línea 20.

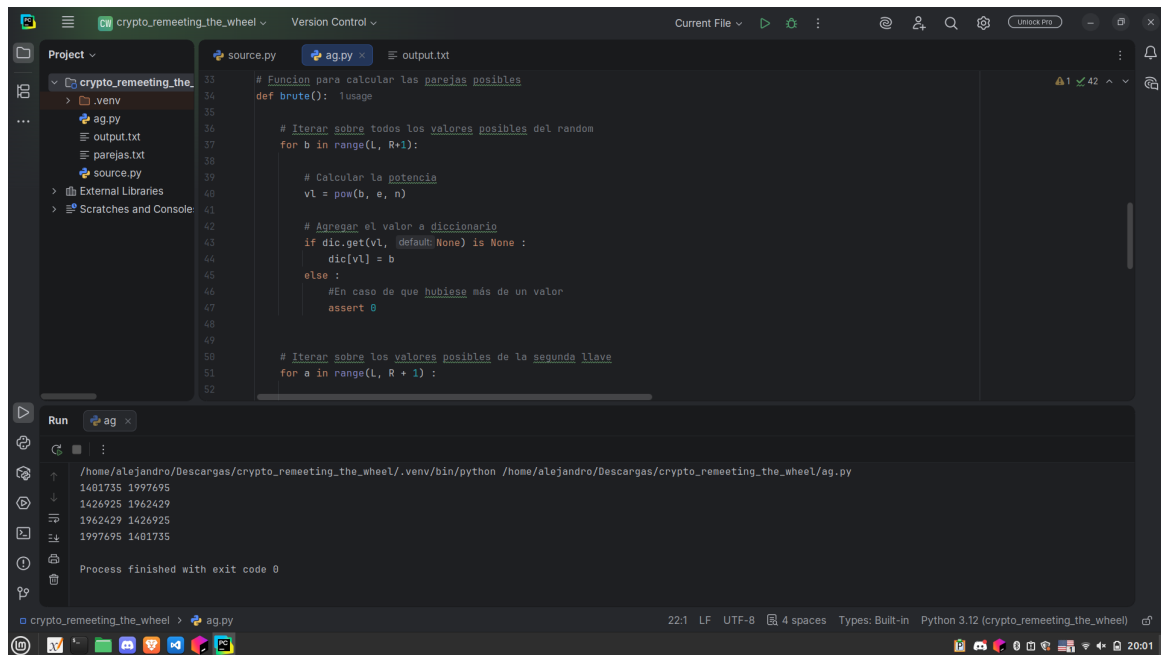
```
33 # Funcion para calcular las parejas posibles
34 def brute(): 1usage
35
36 # Iterar sobre todos los valores posibles del random
37 for b in range(L, R+1):
38
39     # Calcular la potencia
40     vl = pow(b, e, n)
41
42     # Agregar el valor a diccionario
43     if dic.get(vl, default: None) is None :
44         dic[vl] = b
45     else :
46         #En caso de que hubiese más de un valor
47         assert 0
48
49
50 # Iterar sobre los valores posibles de la segunda llave
51 for a in range(L, R + 1) :
52
53     # Calcular la potencia
54     ae = pow(a, e, n)
55
56     # Calcular la inversa
57     vl = (enc_key * inverse(ae, n)) % n
58
59     # Verificar condición
60     assert (vl * ae) % n == enc_key
61
62     # Verificar la existencia del valor
63     if dic.get(vl, default: None) is not None:
64         print(a, dic[vl])
65 brute()
```

Figura 8: Fragmento de código donde se buscan los valores de k_1 y k_2

En el primer ciclo de la función `brute` en la figura 8 se guarda en un diccionario la llave $b^e \bmod n$ y su valor b para cada valor en el rango definido en la figura 4, verificando que no existan dos valores con la misma llave (línea 45). En el segundo ciclo se buscan todas las parejas (a, b) que cumplan con la ecuación definida en la figura 4, en donde se verifica si existe b por medio de la búsqueda de $b^e \bmod n$

en el diccionario (línea 63).

De esta forma se separa el proceso de búsqueda en dos partes, tal que en la primera se preprocesan valores y en la segunda se calcula la respuesta en base a los valores guardados. “Meet in the Middle”, en este caso, consiste simplemente en dividir el espacio de búsqueda en dos partes, lo cual lo hace más eficiente al pasar de 2^{42} a 2^{21} .



The screenshot shows a code editor with a project named 'crypto_remeeting_the_wheel'. The file explorer on the left shows files: .venv, ag.py, output.txt, parejas.txt, source.py, External Libraries, and Scratches and Console. The main editor displays the code in 'ag.py' with line numbers 33 to 52. The code defines a 'brute()' function that iterates over possible values 'b' and 'a' to find pairs that result in the same value. The output.txt file shows the results of the script execution.

```
33 # Funcion para calcular las parejas posibles
34 def brute(): usage
35
36 # Iterar sobre todos los valores posibles del random
37 for b in range(L, R+1):
38
39     # Calcular la potencia
40     vl = pow(b, e, n)
41
42     # Agregar el valor a diccionario
43     if dic.get(vl, default: None) is None :
44         dic[vl] = b
45     else :
46         #En caso de que hubiese más de un valor
47         assert 0
48
49
50 # Iterar sobre los valores posibles de la segunda llave
51 for a in range(L, R + 1) :
```

Run ag x

```
/home/alejandra/Descargas/crypto_remeeting_the_wheel/.venv/bin/python /home/alejandra/Descargas/crypto_remeeting_the_wheel/ag.py
1401735 1997695
1426925 1962429
1962429 1426925
1997695 1401735
Process finished with exit code 0
```

Figura 9: Parejas candidatas obtenidas

Se puede observar que en total se encuentran cuatro parejas candidatas; sin embargo, todas representan la misma llave porque el producto de cada pareja es el mismo. De esta forma, se encontró la llave correspondiente del algoritmo AES con un script que dura unos cuantos segundos.

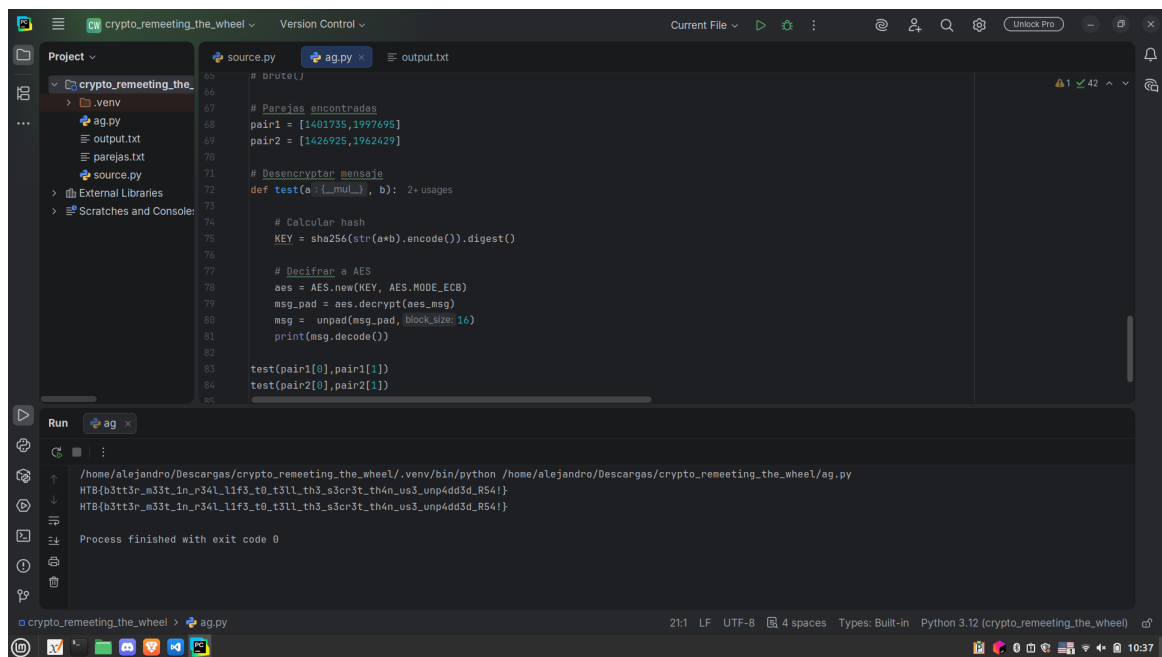
```

67     # Parejas encontradas
68     pair1 = [1401735,1997695]
69     pair2 = [1426925,1962429]
70
71     # Desencryptar mensaje
72     def test(a: {__mul__}, b): 2+ usages
73
74         # Calcular hash
75         KEY = sha256(str(a*b).encode()).digest()
76
77         # Decifrar a AES
78         aes = AES.new(KEY, AES.MODE_ECB)
79         msg_pad = aes.decrypt(aes_msg)
80         msg = unpad(msg_pad, block_size: 16)
81         print(msg.decode())
82
83     test(pair1[0],pair1[1])
84     test(pair2[0],pair2[1])
85

```

Figura 10: Fragmento de código donde se prueban las parejas posibles de k_1 y k_2

En la figura 10 se puede observar la definición de la función `test()` para probar las parejas candidatas. Primero en las líneas 68 y 69 se “alambran” el conjunto de parejas candidatas diferentes. En la línea 75, se calcula el hash de 256 bits del producto de la pareja. En las líneas 78 y 79, se define el `AES` con el hash de la llave y se descifra los bytes del mensaje, respectivamente. En la línea 80, se aplica el inverso del padding y en la línea 81 se imprime el mensaje obtenido. Finalmente, en las líneas 83 y 84 se llama a la función con cada pareja candidata.



```
65 # brutefor
66
67 # Pares encontrados
68 pair1 = [1401735, 1997695]
69 pair2 = [1426925, 1962429]
70
71 # Descifrar mensaje
72 def test(a: int, b: int):
73     # Calcular hash
74     KEY = sha256(str(a+b).encode()).digest()
75
76     # Decifrar a AES
77     aes = AES.new(KEY, AES.MODE_ECB)
78     msg_pad = aes.decrypt(aes_msg)
79     msg = unpad(msg_pad, block_size=16)
80     print(msg.decode())
81
82 test(pair1[0], pair1[1])
83 test(pair2[0], pair2[1])
84
85
```

Run

```
/home/alejandros/Descargas/crypto_remeeting_the_wheel/.venv/bin/python /home/alejandros/Descargas/crypto_remeeting_the_wheel/ag.py
HTB{b3tt3r_m33t_1n_r34l_l1f3_t0_t3ll_th3_s3cr3t_th4n_us3_unp4dd3d_R541}
HTB{b3tt3r_m33t_1n_r34l_l1f3_t0_t3ll_th3_s3cr3t_th4n_us3_unp4dd3d_R541}
```

Process finished with exit code 0

Figura 11: Posibles Mensajes obtenidos tras la ejecución de la figura 10

En la figura 11, se puede observar los resultados tras la ejecución del código. A pesar de ser dos parejas distintas, ambas descifran el mismo mensaje, pues el producto de los valores que conforman las parejas dan el mismo resultado (2 800 239 000 825). Por lo cual el hash y el proceso de descifrado se realizaron con el mismo valor y por ende, descifrando el mismo mensaje.

7. Evidencia de resolución

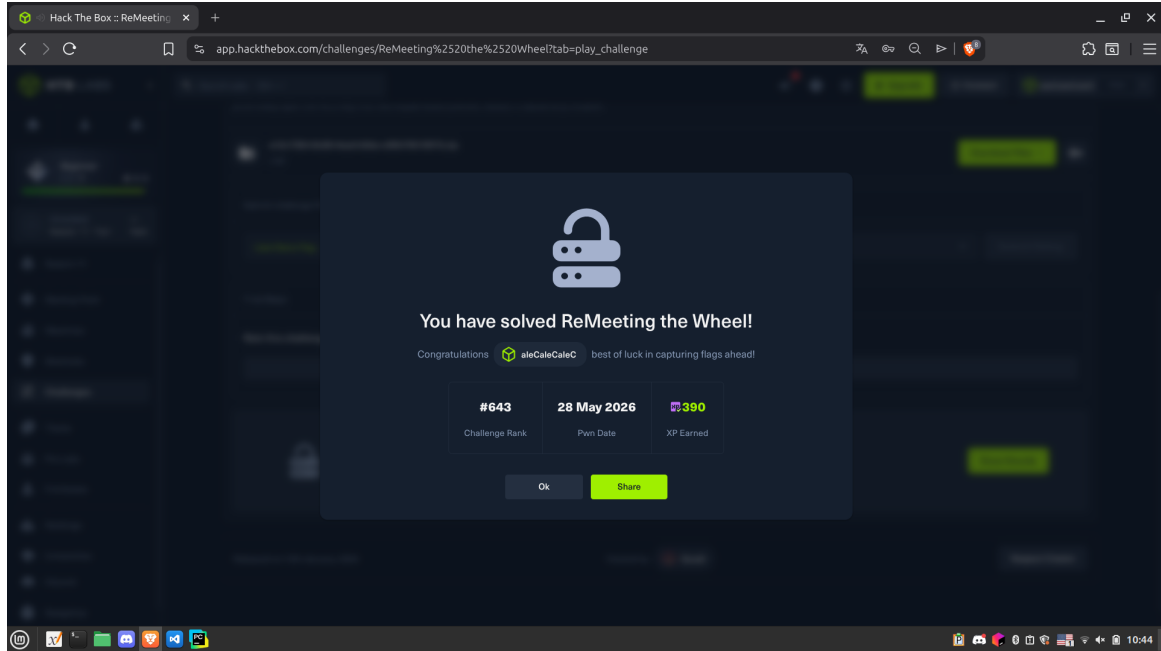


Figura 12: Resolución del challenge

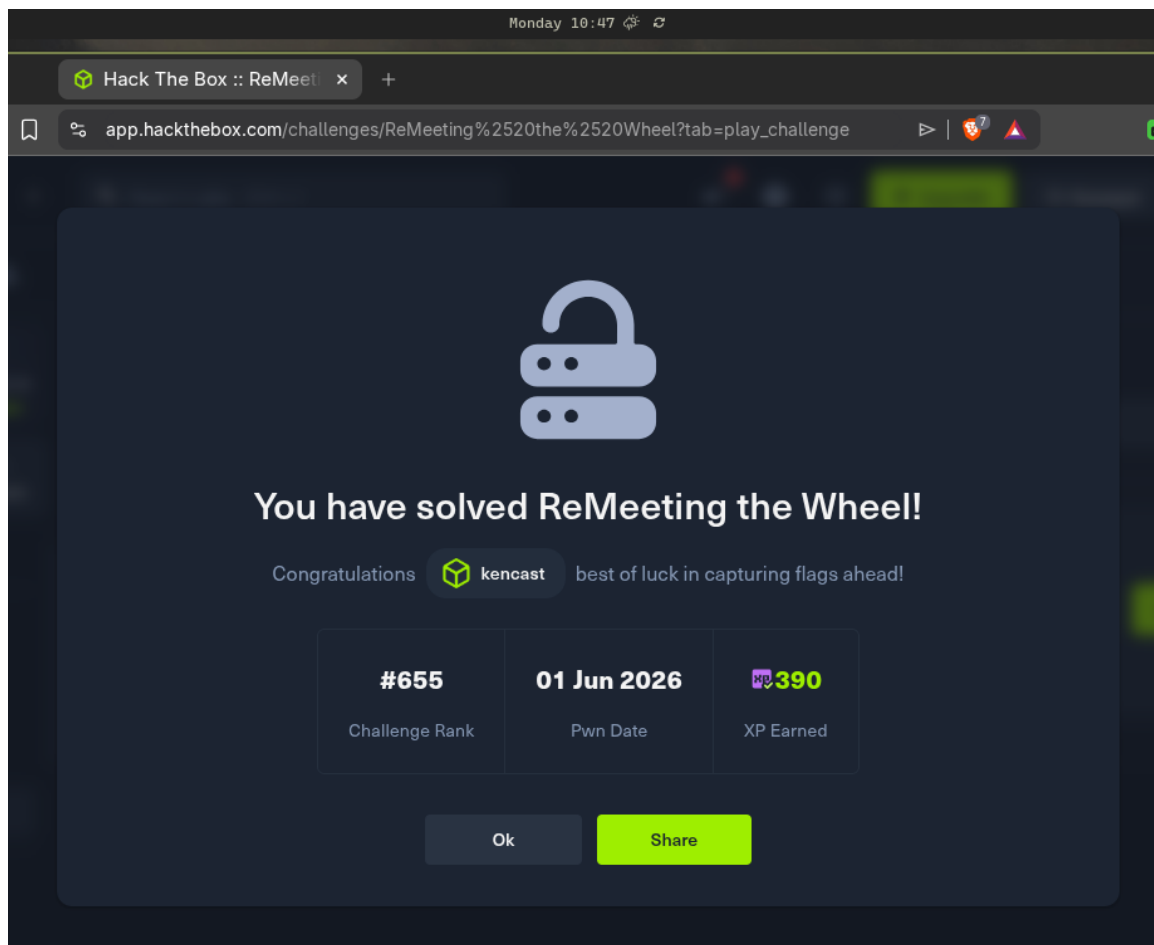


Figura 13: Resolución del challenge

8. Remediación

La vulnerabilidad explotada en este documento puede ser mitigada, evitando generar la llave de [AES](#) mediante la multiplicación de dos números de 2^{20} . Lo ideal es que si se va a usar una llave entre 2^{40} y 2^{42} se genere con un muy buen random por ejemplo el que ofrece la biblioteca **Crypto** conocido como `getRandomValues`, el cual obtiene valores aleatorios bastante seguros. Específicamente, `randint` no es un buen generador de números aleatorios para criptografía [16]. Aun así, se recomienda generar la llave directamente de tamaño 2^{256} o un valor cercano, ya que, además de ahorrarse el proceso de hash, una llave de 2^{40} sigue siendo pequeño para el creciente poder computacional y la posibilidad de paralelizar operaciones.

Adicionalmente, en un entorno de producción real, se recomienda hacer un cambio de [RSA-1024](#) a [RSA-2048](#) o más. [RSA-1024](#) ya no es recomendado y se considera que se puede llegar a factorizar la clave pública $n = pq$ con la capacidad de cómputo actual. Para realizar este cambio, es simplemente llamar a la clase `RSAGen` con el parámetro 2048 en vez de 1024, como se observa en la figura 5.

Por otro lado, es necesario verificar que las claves que se generan para estos algoritmos sean seguras y lo suficientemente aleatorias. Para este caso la biblioteca **Crypto** fue una muy buena opción.

Las buenas prácticas relacionadas con los principios de seguridad desde el diseño son las siguientes [17]:

- **Definir y usar estándares criptográficos:** es crítico usar los mecanismos criptográficos de forma correcta y segura, entendiendo cómo estos funcionan y definiendo las formas de proteger y manejar las llaves de cifrado y certificados. Esto incluye también usar bibliotecas que tengan una seguridad comprobada para los procedimientos criptográficos y matemáticos.
- **Requerir el uso de funciones y herramientas con seguridad comprobada:** asegurarse de que los equipos de desarrollo usen soluciones bien establecidas y de seguridad comprobada. Manteniéndose al día con los cambios y las versiones de las herramientas. Por ejemplo, usando una versión segura de RSA.

9. Lecciones aprendidas

Con este challenge se pudo profundizar y lograr una buena comprensión de los algoritmos [RSA](#) y [AES](#), así como de las vulnerabilidades que estos pueden tener si no se implementan de forma correcta, y también los estándares y prácticas recomendadas. En general, se pudo observar e indagar un poco sobre los métodos criptográficos y prácticas que se usan para una transmisión segura de información, en especial, cómo se combinan los algoritmos de cifrado simétricos y asimétricos para transmitir información de forma segura y eficiente. Adicionalmente, se estudió la biblioteca [Crypto](#) y sus diferentes funciones, la cual que ofrece poderosas funciones ideales para el cifrado de información. Además, se observó que técnicas popularizadas en la programación competitiva también son útiles (gracias a su eficiencia, en la mayoría de casos) para resolver ciertos problemas en criptografía. Lo que se haría diferente en este challenge sería notar de forma más directa algunas propiedades matemáticas que permitían descifrar el mensaje.

10. Acrónimos

[RSA](#) Rivest, Shamir y Adleman

[AES](#) Advanced Encryption Standard

[HTB](#) Hack The Box

11. Recursos consultados

- [1] Información sobre el algoritmo RSA: https://en.wikipedia.org/wiki/RSA_cryptosystem
- [2] Módulo de Python usado en el script: <https://www.pycrypto.org/api/current/Crypto.Util.number-module.html>
- [3] Explicación del algoritmo RSA: <https://www.geeksforgeeks.org/computer-networks/rsa-algorithm-cryptography/>

- [4] Explicación del algoritmo AES: https://es.wikipedia.org/wiki/Advanced_Encryption_Standard
- [5] Funcionamiento de la biblioteca de AES <https://pycryptodome.readthedocs.io/en/latest/src/cipher/aes.html>
- [6] Funcionamiento de los diccionarios en python <https://ellibrodepython.com/diccionarios-en-python>
- [7] Explicación de las funciones de la biblioteca Crypto Util, como padding y long to bytes. <https://pycryptodome.readthedocs.io/en/latest/src/util/util.html>
- [8] Funcionamiento de pow en python https://www.w3schools.com/python/ref_func_pow.asp
- [9] Como se puede convertir de bytes a un string en python <https://realpython.com/convert-python-bytes-to-strings/>
- [10] Como usar la función fromhex() de python <https://www.geeksforgeeks.org/python/bytes-fromhex-method-python/>
- [11] Como funciona el método de randint en python https://www.w3schools.com/python/ref_random_randint.asp
- [12] Convertir de hexadecimal a decimal <https://www.rapidtables.org/convert/number/hex-to-decimal.html>
- [13] Common Weakness Enumeration: <https://cwe.mitre.org/index.html>
- [14] Common Attack Pattern Enumeration and Clasification: <https://capec.mitre.org/index.html>
- [15] Common Vulnerabilities and Exposures: <https://www.cve.org/About/Overview>
- [16] Explicación de la función para generar números aleatorios con Crypto <https://developer.mozilla.org/en-US/docs/Web/API/Crypto/getRandomValues>
- [17] Principios de seguridad del software desde el diseño: <https://www.microsoft.com/en-us/securityengineering/sdl/practices>